BÁO CÁO THỰC HÀNH LAB-02

EMBEDDED LINUX SYSTEMS

Viết Device Driver và Quản lý File System

Họ và tên: Lê Tấn Minh

MSSV: SE200684

Nhóm: Chưa cung cấp

Ngày thực hiện báo cáo: 03/11/2026

Môi trường thực hành: Ubuntu host, ARM Versatile Express A9 trên QEMU, Linux kernel 5.15.0.

1. Thông tin sinh viên và môi trường

Thông tin	Giá trị
Họ và tên	Lê Tấn Minh
MSSV	SE200684
Nhóm	Chưa cung cấp
Ngày thực hiện	03/11/2026
Host	Ubuntu Linux x86_64
Kernel mục tiêu	Linux 5.15.0, ARMv7, armv7l
Cross compiler	arm-linux-gnueabihf-gcc 15.2.0
Mô hình QEMU	vexpress-a9, CPU cortex-a9, RAM 512 MiB
Initramfs	output/initramfs_lab2.cpio.gz
Driver	lab2_driver.ko; character device major 240
NAND simulator	128 MiB, erase block 16 KiB, page 512 B

Mục tiêu thực hành là xây dựng character device driver, trao đổi dữ liệu qua /dev/lab2, cung cấp thông tin qua procfs/sysfs, kiểm tra JFFS2 trên NAND giả lập và tự động nạp module khi khởi động initramfs.

2. Sơ đồ kiến trúc driver

Luồng xử lý tổng quát khi userspace truy cập character device:

Tầng	Vai trò
Userspace application	Chương trình test mở /dev/lab2 và gọi read()/write().
System call	Chuyển yêu cầu từ userspace vào kernel.
VFS	Tìm file/device và chuyển thao tác tới file_operations.
file_operations	Các callback driver xử lý open, read, write, release.
device_buffer	Vùng đệm kernel chứa dữ liệu của driver.
procfs / sysfs	Cung cấp trạng thái/thông tin driver cho userspace đọc.

Luồng dữ liệu

```
WRITE:
Userspace buffer
    -> write() system call
    -> VFS / file_operations
    -> copy_from_user()
    -> device_buffer (kernel space)
    -> thông tin trạng thái qua /proc/lab2_info và /sys/class/lab2_class/lab2/

READ:
device_buffer (kernel space)
    -> copy_to_user()
    -> VFS / read() system call
    -> Userspace buffer
```

Các thuộc tính sysfs được kiểm tra trong bài gồm buffer_len, open_count và last_data. Driver được nạp trong initramfs; log boot cho thấy major number 240 và thông báo tạo procfs/sysfs.

3. Log thực hành

3.1. Khởi động kernel và tự động nạp driver

```
Linux version 5.15.0 (minhltse200684@minhltse200684-Dell-15-DC15250) (arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, GNU ld (GNU Bi
Kernel command line: console=ttyAMA0,115200 rdinit=/init
```

3.2. Kiểm thử character device và procfs/sysfs

Log dưới đây là bản tổng hợp từ kết quả kiểm thử thực tế được người thực hành ghi lại; không phải bản ghi terminal thô. Kết quả test đã báo cáo: cả 6 kiểm tra đều PASS.

```
LAB-02 AUTOMATED DRIVER TEST

Kernel: Linux 5.15.0, ARMv7

[TEST 1] Character device
PASS: /dev/lab2 exists

[TEST 2] Write data
PASS: write succeeded

[TEST 3] Read data
Read data: Hello LAB2 Part D
PASS: read data matches

[TEST 4] Procfs
Driver: lab2
Major: 240
Buffer size: 1024
Buffer length: 17
Overwrites: 1
Open count: 0
Last data: Hello LAB2 Part D
PASS: procfs is readable

[TEST 5] Sysfs
buffer_len: 17
open_count: 0
last_data: Hello LAB2 Part D
PASS: sysfs attributes are readable

[TEST 6] Kernel module
PASS: lab2_driver is loaded

RESULT: ALL TESTS PASSED
```

3.3. Xóa và ghi ảnh JFFS2 lên NAND giả lập

```
===== flash_erase =====
Erasing 131072 Kibyte @ 0 --  0 % complete
Erasing 131072 Kibyte @ 0 -- 100 % complete
===== nandwrite =====
Writing data to block 0 at offset 0x0
```

Thông tin thiết bị lấy từ mtdinfo: NAND simulator partition 0, dung lượng 128 MiB, erase block 16 KiB, page 512 bytes, OOB 16 bytes. Đây là NAND giả lập của host, không phải thiết bị flash vật lý 32 MiB.

3.4. Mount và kiểm tra JFFS2

```
===== MTD INFORMATION =====
mtd1
Name:                           NAND simulator partition 0
Type:                           nand
Eraseblock size:                16384 bytes, 16.0 KiB
Amount of eraseblocks:          8192 (134217728 bytes, 128.0 MiB)
Minimum input/output unit size: 512 bytes
Sub-page size:                  256 bytes
OOB size:                       16 bytes
Character device major/minor:   90:2
Bad blocks are allowed:         true
Device is writable:             true


===== ERASE LOG =====

Erasing 131072 Kibyte @ 0 --  0 % complete
Erasing 131072 Kibyte @ 0 -- 100 % complete

===== WRITE LOG =====
Writing data to block 0 at offset 0x0

===== JFFS2 FILES =====
```

```
total 4
drwxr-xr-x 3 root          root             0 Jan  1  1970 .
drwxrwxr-x 4 minhltse200684 minhltse200684 4096 Nov  3 14:14 ..
-rw-r--r-- 1 root          root            35 Nov  3 14:17 persistence_after_mount.txt
-rw-rw-r-- 1 minhltse200684 minhltse200684   28 Nov  3 14:14 persistence_test.txt
-rw-rw-r-- 1 minhltse200684 minhltse200684   36 Nov  3 14:14 student_info.txt

===== PERSISTENCE TEST =====
LAB2 JFFS2 persistence test
MSSV: SE200684
Student: Le Tan Minh
Persistence verified after remount
```

JFFS2 mount thành công qua /dev/mtdblock1. Đọc được persistence_test.txt và student_info.txt. persistence_after_mount.txt được đọc sau thao tác unmount/mount lại, xác nhận dữ liệu vẫn tồn tại trong lần kiểm tra này.

3.5. Build log

```
‘.’
‘./init’
‘./tmp’
‘./lib’
‘./lib/modules’
‘./lib/modules/5.15.0’
‘./lib/modules/5.15.0/lab2_driver.ko’
‘./lib/arm-linux-gnueabihf’
‘./proc’
‘./dev’
‘./bin’
‘./bin/sleep’
‘./bin/setserial’
‘./bin/mount’
‘./bin/ionice’
‘./bin/sh’
‘./bin/link’
‘./bin/cat’
‘./bin/base32’
‘./bin/dd’
‘./bin/base64’
‘./bin/nice’
‘./bin/hush’
‘./bin/chown’
‘./bin/mkdir’
‘./bin/chgrp’
‘./bin/mknod’
‘./bin/cpio’
‘./bin/stat’
‘./bin/sync’
‘./bin/fgrep’
‘./bin/mt’
‘./bin/getopt’
‘./bin/kill’
‘./bin/ls’
‘./bin/stty’
‘./bin/df’
‘./bin/mv’
‘./bin/dumpkmap’
‘./bin/arch’
‘./bin/zcat’
‘./bin/true’
‘./bin/egrep’
‘./bin/setpriv’
‘./bin/date’
‘./bin/busybox’
‘./bin/more’
‘./bin/scriptreplay’
‘./bin/rev’
‘./bin/kbd_mode’
‘./bin/chmod’
‘./bin/dmesg’
‘./bin/vi’
‘./bin/touch’
‘./bin/pwd’
‘./bin/iostat’
‘./bin/grep’
‘./bin/watch’
‘./bin/linux32’
‘./bin/linux64’
```

‘./bin/makemime’
‘./bin/usleep’
‘./bin/gzip’
‘./bin/fsync’
‘./bin/setarch’
‘./bin/run-parts’
‘./bin/printenv’
‘./bin/umount’
‘./bin/rpm’
‘./bin/cttyhack’
‘./bin/pidof’
‘./bin/hostname’
‘./bin/lzop’
‘./bin/su’
‘./bin/ping6’
‘./bin/gunzip’
‘./bin/dnsdomainname’
‘./bin/netstat’
‘./bin/cp’
‘./bin/echo’
‘./bin/tar’
‘./bin/false’
‘./bin/fdflush’
‘./bin/chattr’
‘./bin/ash’
‘./bin/mpstat’
‘./bin/ping’
‘./bin/fatattr’
‘./bin/resume’
‘./bin/rm’
‘./bin/sed’
‘./bin/ps’
‘./bin/rmdir’
‘./bin/mountpoint’
‘./bin/lsattr’
‘./bin/uname’
‘./bin/ln’
‘./bin/ipcalc’
‘./bin/ed’
‘./bin/mktemp’
‘./bin/login’
‘./bin/pipe_progress’
‘./bin/reformime’
‘./bin/conspy’
‘./sys’
‘./usr’
‘./usr/bin’
‘./usr/bin/diff’
‘./usr/bin/sha512sum’
‘./usr/bin/truncate’
‘./usr/bin/lpq’
‘./usr/bin/openvt’
‘./usr/bin/logger’
‘./usr/bin/comm’
‘./usr/bin/runsvdir’
‘./usr/bin/flock’
‘./usr/bin/wall’
‘./usr/bin/ts’
‘./usr/bin/whois’
‘./usr/bin/cmp’
‘./usr/bin/strings’
‘./usr/bin/unlzma’
‘./usr/bin/ftpput’
‘./usr/bin/expr’
‘./usr/bin/setfattr’
‘./usr/bin/nslookup’
‘./usr/bin/telnet’
‘./usr/bin/shuf’
‘./usr/bin/lsusb’
‘./usr/bin/ipcrm’
‘./usr/bin/svok’
‘./usr/bin/hexedit’
‘./usr/bin/hostid’
‘./usr/bin/basename’
‘./usr/bin/taskset’
‘./usr/bin/udhcpc6’
‘./usr/bin/fallocate’
‘./usr/bin/pwdx’
‘./usr/bin/unzip’
‘./usr/bin/wget’
‘./usr/bin/lpr’

‘./usr/bin/uuencode’
‘./usr/bin/last’
‘./usr/bin/install’
‘./usr/bin/killall’
‘./usr/bin/passwd’
‘./usr/bin/setkeycodes’
‘./usr/bin/seq’
‘./usr/bin/sha3sum’
‘./usr/bin/unix2dos’
‘./usr/bin/unlink’
‘./usr/bin/vlock’
‘./usr/bin/od’
‘./usr/bin/sum’
‘./usr/bin/script’
‘./usr/bin/timeout’
‘./usr/bin/uniq’
‘./usr/bin/deallocvt’
‘./usr/bin/free’
‘./usr/bin/test’
‘./
... [đã rút gọn để phù hợp báo cáo] ...

4. So sánh filesystem

Tiêu chí	ext4	JFFS2	cramfs
Writable	Có	Có	Chỉ đọc
Wear leveling	Không tự quản lý NAND thô; thường cần lớp block/FTL phù hợp	Không có wear leveling mạnh; ghi log phân tán trong vùng flash	Không ghi trực tiếp lên filesystem chỉ đọc
Compression	Không phải đặc điểm bắt buộc; có thể hỗ trợ tùy cấu hình	Có hỗ trợ nén dữ liệu	Filesystem nén, chỉ đọc
Boot time	Thường nhanh trên thiết bị block phù hợp; phụ thuộc dung lượng và kiểm tra filesystem	Có thể cần quét log flash khi mount; phụ thuộc kích thước flash	Mount thường nhanh do chỉ đọc và cấu trúc đơn giản
Thiết bị phù hợp	SD/eMMC/SSD và thiết bị có block device	NAND/NOR flash nhỏ đến vừa, hệ thống nhúng	Firmware/root filesystem bất biến, cần tiết kiệm dung lượng
OTA / cập nhật	Thuận tiện trên block storage; cần quản lý phân vùng và tính toàn vẹn	Có thể ghi cập nhật nhưng cần thiết kế vùng dự phòng và xử lý mất điện	Không cập nhật trực tiếp tại chỗ; thường thay image/phân vùng

Khuyến nghị cho thiết bị IoT có 32 MiB NAND và cần OTA

Với NAND thô, không nên coi ext4 là lựa chọn trực tiếp nếu không có lớp block translation phù hợp. JFFS2 có thể dùng cho vùng flash nhỏ, ghi được và có hỗ trợ nén, nhưng cần tính đến thời gian quét khi mount, giới hạn dung lượng và tuổi thọ flash. Với thiết kế sản phẩm, cần đánh giá thêm UBI/UBIFS cho NAND, vốn được thiết kế để làm việc với đặc tính NAND và quản lý erase block tốt hơn. Dù chọn filesystem nào, OTA nên có cơ chế kiểm tra chữ ký/toàn vẹn, rollback và phân vùng A/B hoặc vùng recovery nếu ngân sách 32 MiB cho phép. Kết luận cuối cùng phụ thuộc kích thước firmware và dung lượng thực tế dành cho hai image.

5. Trả lời câu hỏi phân tích

Câu 1. Vì sao không dùng memcpy() giữa kernel buffer và user buffer?

Kernel space và user space có không gian địa chỉ ảo, quyền truy cập và ánh xạ bộ nhớ khác nhau. Con trỏ từ userspace không thể được xem như một con trỏ kernel hợp lệ. memcpy() chỉ sao chép byte giữa hai vùng nhớ mà nó giả định truy cập trực tiếp được; dùng nó với địa chỉ userspace có thể gây lỗi truy cập kernel, làm sập hệ thống hoặc truy cập bộ nhớ không hợp lệ. copy_from_user() sao chép dữ liệu từ user buffer vào kernel buffer và kiểm tra khả năng truy cập địa chỉ nguồn; copy_to_user() thực hiện chiều ngược lại. Hai hàm có thể trả về số byte chưa sao chép được, nên driver cần xử lý lỗi thay vì giả định luôn thành công.

Câu 2. Vì sao cần DEFINE_MUTEX(lab2_mutex)?

Mutex bảo vệ vùng dữ liệu dùng chung khi nhiều tiến trình cùng gọi read()/write(). Ví dụ, hai tiến trình đồng thời ghi các chuỗi A và B: nếu mỗi thao tác cập nhật buffer_len và device_buffer không được đồng bộ, dữ liệu có thể bị ghi đè, kết quả đọc chứa nội dung không nhất quán, hoặc độ dài buffer không khớp với dữ liệu. Đây là race condition do thứ tự xen kẽ giữa các thao tác. Mutex bảo đảm chỉ một luồng thực thi trong vùng tới hạn tại một thời điểm; mọi đường truy cập liên quan phải tuân theo cùng quy tắc khóa/mở khóa để có hiệu quả.

Câu 3. So sánh procfs và sysfs

procfs (/proc) chủ yếu cung cấp thông tin về tiến trình và trạng thái hệ thống; nó cũng có các giao diện lịch sử của kernel. sysfs (/sys) biểu diễn đối tượng kernel, thiết bị, driver và thuộc tính của chúng theo cấu trúc có tổ chức. Kernel thường khuyến nghị dùng sysfs cho thuộc tính của thiết bị/driver thay vì thêm các file tùy ý vào /proc, vì sysfs gắn thuộc tính với đối tượng kernel và có quy ước ABI rõ ràng. Trong bài lab, /proc/lab2_info phù hợp để hiển thị bản tóm tắt trạng thái driver; các giá trị riêng như buffer_len, open_count và last_data phù hợp với các attribute trong /sys/class/lab2_class/lab2/.

Câu 4. Vì sao NAND cần erase block và JFFS2 dùng log-structured?

NAND flash không thể ghi đè tùy ý lên dữ liệu đã lập trình như sector trên một block device thông thường. Các bit chỉ có thể chuyển theo chiều được công nghệ flash cho phép khi lập trình; để tái sử dụng vùng đã ghi, phải xóa cả erase block. Vì vậy, thao tác cập nhật dữ liệu có thể cần ghi phiên bản mới vào vùng khác rồi đánh dấu dữ liệu cũ là không còn hiệu lực. JFFS2 tổ chức dữ liệu theo dạng log trên flash, ghi node mới thay vì cập nhật tại chỗ, rồi thu gom/xóa các block khi cần. Trong bài thực hành, ảnh JFFS2 được tạo với erase block 16 KiB, ghi lên NAND giả lập, mount qua /dev/mtdblock1 và kiểm tra dữ liệu sau khi mount lại.

Câu 5. Nếu insmod thất bại trong rcS thì sao?

Việc hệ thống có tiếp tục boot hay không phụ thuộc cách rcS và init xử lý mã thoát. Nếu script không dừng khi một lệnh thất bại, các lệnh sau vẫn có thể chạy; nếu dùng cơ chế dừng ngay hoặc chính sách init yêu cầu, quá trình có thể bị ảnh hưởng. Nên kiểm tra kết quả của insmod, ghi cảnh báo vào console/log và xác minh thiết bị /dev/lab2 đã xuất hiện. Nếu driver là thành phần bắt buộc để hệ thống hoạt động an toàn, script có thể chuyển sang recovery hoặc dừng với thông báo rõ ràng; nếu không bắt buộc, có thể tiếp tục boot ở chế độ hạn chế. Ví dụ xử lý có kiểm tra:

```
if insmod /lib/modules/5.15.0/lab2_driver.ko; then
echo '[LAB2] Driver loaded' > /dev/console
else
echo '[LAB2] WARNING: driver load failed' > /dev/console
fi
```

Log boot thực tế của bài này có các dòng '[LAB2] Driver loaded successfully' và '[LAB2] Startup script finished', cho thấy module đã được nạp trong lần chạy được ghi lại.

6. Nhận xét và khó khăn

Khó khăn 1: Mount JFFS2 trực tiếp từ /dev/mtd1 thất bại

Lỗi quan sát: mount báo '/dev/mtd1 is not a block device'. Kiểm tra mtdinfo cho thấy /dev/mtd1 là MTD character device. Sau khi nạp mtdblock, hệ thống tạo /dev/mtdblock1 và lệnh 'mount -t jffs2 /dev/mtdblock1 mtd/mountpoint' hoạt động; các file mẫu đọc được.

Khó khăn 2: Đồng bộ driver giữa các thư mục

Kiểm tra SHA-256 cho thấy driver/lab2_driver.ko và bản trong initramfs giống nhau, nhưng rootfs/lab2_driver.ko có hash khác. Cần đồng bộ bản rootfs với driver đã kiểm thử và xác minh lại hash trước khi đóng gói để tránh nộp nhầm phiên bản.

Khó khăn 3: Dung lượng NAND giả lập khác tình huống đề bài

NAND simulator trên host báo 128 MiB, erase block 16 KiB và page 512 bytes. Đề bài yêu cầu phân tích tình huống IoT 32 MiB NAND; hai số liệu này phục vụ hai mục đích khác nhau và không được đánh đồng trong báo cáo.

Kết luận

Các log hiện có xác nhận kernel 5.15.0 chạy trên QEMU, initramfs khởi động, driver được nạp, các test character device/procfs/sysfs đã báo PASS trong kết quả ghi nhận và JFFS2 đọc được dữ liệu sau thao tác mount lại. Báo cáo sử dụng log và thông số hiện có; những nội dung chưa có bằng chứng riêng được ghi rõ thay vì suy diễn thành kết quả thành công.

Phụ lục: Tệp bằng chứng

logs/boot_log.txt
logs/driver_test.txt
logs/mtd_erase.log
logs/mtd_write.log
logs/mtd_jffs2.txt
logs/cpio_build.log
driver/lab2_driver.c
driver/Makefile
output/initramfs_lab2.cpio.gz
mtd/lab2_rootfs.jffs2